

Xinu `xinu-rpi4` User's Manual

Embedded Xinu for the Raspberry Pi 4 (BCM2711 / Cortex-A72, AArch64)

Embedded Xinu port for the Raspberry Pi 4 (BCM2711 / Cortex-A72, AArch64). The same source tree also cross-builds for the QEMU `virt` machine. This manual is written for **operators** — for the development roadmap see `README.md`.

Contents

1	Targets and Build Artefacts	1
1.1	Feature Matrix (read this first!)	1
1.2	Boot Sequence (the unusual bits)	2
2	What You Need	2
2.1	Hardware (for a real board)	2
2.2	Host software	2
3	Build	2
3.1	Basic commands	3
3.2	Toolchain auto-detection	3
3.3	Key Make variables	3
3.4	Per-target macros (passed via <code>CFLAGS</code>)	3
3.5	Linker scripts	4
4	Writing the SD Card	4
4.1	Pi 4	4
4.2	Manual copy (e.g. external SD reader)	4
4.3	What else must be on the card	4
5	Serial Console	4
6	Shell Command Reference	5
6.1	<code>procdemo</code> — real context switch	5
6.2	<code>pingpong</code> vs <code>procdemo</code>	6
6.3	On-device tooling (filesystem, C JIT, actors, LLM)	6
6.4	Virtual memory (<code>vmtest</code> / <code>vmdemand</code>)	6
7	Networking (Pi 4)	6
7.1	Ping from a Mac	7
7.2	DHCP / TCP status	7
7.3	WiFi (BCM43455)	7

7.4	Mesh networking: multiple Xinu nodes (MANET ad-hoc)	7
7.5	HTTP control plane & remote shell	8
8	Running under QEMU	9
9	Important config.txt Lines	9
10	USB / Input Device Support	9
11	Troubleshooting	10
12	Directory Layout (operator view)	10
13	Recipe Book	10
14	References	11
15	License	11

1 Targets and Build Artefacts

Target	Board / machine	Output image	UART0 base
pi4	Raspberry Pi 4 (BCM2711)	kernel8.img	0xFE201000
qemu	QEMU -M virt -cpu cortex-a76	kernel_virt.img	0x09000000

The load address is **0x80000** for Pi 4 and **0x40080000** for QEMU. Both images come from one source tree, distinguished by `-D<TARGET>` macros and a per-target linker script.

1.1 Feature Matrix (read this first!)

Not every feature works on every target. Several subsystems run only on real hardware (Pi 4):

Subsystem	Pi 4	QEMU	Notes
PL011 UART0 shell	✓	✓	The common substrate
Cooperative scheduler (S0)	✓	✓	procdemo / pingpong
Preemptive scheduler (S1)	✓	WIP	100 Hz timer tick works on Pi 4 (ticks)
HDMI framebuffer	✓	—	Usable on Pi 4
Window manager	✓	—	1280×960 virtual desktop, Pi 4 only
SD card + FAT32	✓	—	EMMC driver is BCM2711-specific
Ethernet (GENET)	✓	—	NET-E done (BCM2711 GENET)
USB (keyboard / HID)	✓	—	Via VL805 PCIe xHCI (USB-2.0 ports)
DHCP / TCP	✗	—	Code present but dispatch OFF (§7.4)

! The source tree contains `device/genet/`, `device/sd/`, `device/usb/xhci/` etc., initialised against **Pi 4 (BCM2711) register addresses**. The `-D<BASE>` macros (`GENET_BASE`, ...) are defined only for the Pi 4 build; the QEMU build leaves them undefined so the corresponding code is linked out or no-ops.

1.2 Boot Sequence (the unusual bits)

loader/boot.S does more than a textbook bare-metal stub:

1. **Mandatory Linux ARM64 Image header.** The Pi 4 EEPROM bootloader can refuse to jump into the kernel unless it sees the magic "ARM\64" at file offset 0x38. `boot.S` places a 64-byte header right at `_start`. Symptom of a missing header: the firmware rainbow pattern stays on HDMI and your code never runs.
2. **Pin secondary cores with MPIDR_EL1.** The boot core continues; cores 1/2/3 enter a `wfe` park loop until S0/S1 wakes them.
3. **Set up SP using the leex convention.** `sp = _start (= 0x80000)` — the kernel base address doubles as the initial stack pointer.
4. **Preserve the DTB pointer.** Firmware passes the device tree physical address in `x0`; the stub stashes it across BSS clear and writes it back to `.data dtb_addr`.
5. **Zero BSS, then jump to C** (`bl kernel_main`). If `kernel_main` ever returns, drop into the same `wfe` park loop.

The stub does **not** implement an EL2 → EL1 transition (same assumption holds for QEMU). Adding `armstub=` or `kernel_old=1` would change this.

2 What You Need

2.1 Hardware (for a real board)

- Raspberry Pi 4 (BCM2711)
- microSD card with a FAT32 `bootfs` partition (easiest: flash Raspberry Pi OS first, then overwrite `kernel8.img` and `config.txt`)
- 3.3 V USB-serial adapter, 115200 8N1 — header pin 8 (TXD → GPIO14), pin 10 (RXD → GPIO15), and a GND (e.g. pin 6)
- For networking: an Ethernet cable

2.2 Host software

macOS or Linux, plus an AArch64 cross-toolchain (pick one):

```
brew install aarch64-elf-gcc      # GNU
brew install --cask gcc-arm-embedded  # ARM official
```

`qemu-system-aarch64` for the QEMU target (`brew install qemu`), and a serial terminal client (`screen`, `minicom`, `picocom`, ...).

3 Build

Always run `make` from `compile/` — the Makefile uses `VPATH` to gather sources from `../loader/`, `../device/...`, etc.

3.1 Basic commands

```
cd /Users/kodamay/projects/xinu-rpi4/compile

make pi4          # -> compile/kernel8.img      (real Pi 4)
make qemu        # -> compile/kernel_virt.img   (QEMU virt)
make             # = make all = pi4 + qemu
make clean       # remove all objects and images
```

Objects live in per-variant trees (`obj/pi4/`, `obj/qemu/`), so building different targets in sequence only recompiles what differs.

3.2 Toolchain auto-detection

The Makefile looks for AArch64 GCC as `$(GCCPATH)/bin/aarch64-elf-gcc` then `aarch64-none-elf-gcc`. Default `GCCPATH` is `/opt/homebrew`:

```
make pi4 GCCPATH=$HOME/aarch64/arm-gnu-toolchain-...-aarch64-none-elf
make pi4 CROSS=aarch64-linux-gnu-
```

3.3 Key Make variables

Variable	Default	Purpose
GCCPATH	/opt/homebrew	Cross-toolchain install prefix
CROSS	(auto-detected)	Tool prefix (<code>aarch64-elf-</code> , ...)
SDCARD	/Volumes	Parent mount point for <code>make install_*</code>
DEST	\$(SDCARD)/bootfs	Target FAT32 partition

3.4 Per-target macros (passed via CFLAGS)

Macro	Pi 4	QEMU virt
-mcpu	cortex-a72	cortex-a76
UART0_BASE	0xFE201000	0x09000000
MBOX_BASE	0xFE00B880	(undef)
SD_BASE	0xFE340000	—
USB_BASE	0xFE980000	—
GIC_BASE	0xFF840000	—
PCIE_BASE	0xFD500000	—
GENET_BASE	0xFD580000	—
HEAP_END	0x40000000 (1G)	0x50000000
SKIP_MBOX	—	defined (simplify)
KERNEL_NAME	"kernel8.img"	"kernel_virt.img"
BOARD_NAME	"Pi4"	"virt"
SOC_NAME	"BCM2711"	"QEMU"

Common CFLAGS: `-Wall -O2 -ffreestanding -nostdinc -nostdlib -nostartfiles -mgeneral-regs-only`. The flag keeps most code FPU/NEON-free; only `cc.c` and `llm.c` are built with FP enabled (CPACR releases the FPU at EL1).

3.5 Linker scripts

Script	Target	Entry / load address
compile/link.ld	Pi 4	0x80000
compile/link_virt.ld	QEMU virt	0x40080000

Both place `.text.boot` at the start and export `__bss_start` / `__bss_size` (8-byte units), following the leex convention.

4 Writing the SD Card

4.1 Pi 4

```
cd compile
make install_pi4 SDCARD=/Volumes
# -> copies kernel8.img and sdcard/config_pi4.txt to /Volumes/bootfs
```

`make install` is an alias for `install_pi4`.

4.2 Manual copy (e.g. external SD reader)

```
diskutil mount /dev/disk4s1          # -> /Volumes/bootfs
cp compile/kernel8.img /Volumes/bootfs/
cp sdcard/config_pi4.txt /Volumes/bootfs/config.txt
sync
diskutil eject /Volumes/bootfs
```

! Verify the md5 of `kernel8.img` on the card matches `compile/` — booting an old image is the most common time sink.

4.3 What else must be on the card

`kernel8.img` + `config.txt` alone won't boot. The same FAT32 partition also needs the Raspberry Pi OS firmware blobs (`bootcode.bin`, `start4.elf`, `fixup4.dat`, ...). Fastest path: flash Pi OS first, then overwrite only `kernel8.img` and `config.txt`.

5 Serial Console

```
screen /dev/tty.usbserial-XXXX 115200          # macOS
minicom -b 115200 -o -D /dev/tty.usbserial-XXXX # macOS
sudo screen /dev/ttyUSB0 115200                # Linux
```

Exit `screen` with `Ctrl-a k` then `y`. Within 5 seconds of power-on you should see:

```
=====
Xinu Pi4 hello (AArch64, BCM2711, kernel8.img)
PL011 UART0 @ 0xFE201000, 115200 8N1
bootstrap: leex-style stub + xinu-rpi4 main
=====
```

```
Round 1 phase B/U done -- entering interactive shell.
type `help` for the command list.
xinu-pi4$ _
```

6 Shell Command Reference

help or ? always shows the live list. The current command set:

Command	Purpose
help / ?	List registered commands
echo <words>	Echo args back (whitespace-collapsed)
hello	Smoke marker — greeting
mem	Show <code>__bss_start</code> / <code>__bss_end</code> / <code>_end</code>
peek <hex_addr>	Read a 32-bit MMIO word (e.g. <code>peek 0xfe201018</code>)
uptime	Raw <code>CNTPCT_EL0</code> (generic timer counter)
ticks	100 Hz timer tick counter (S1)
ps	Core / EL status
halt	Mask DAIF + PSCI <code>SYSTEM_OFF</code> (QEMU exits cleanly)
reboot	Stub — spins until power-cycle
pwd ls cd mkdir rmdir touch cat write edit rm cp mv tree	In-RAM filesystem (volatile; §6.3)
cc <file.c>	Compile & run a C program on-device (JIT → AArch64; §6.3)
aload / amsg	Load resident AIPL actors / send a message (§6.3)
actordemo / selectdemo	Actor ping-pong / guarded named-message receive (§6.3)
vmtest / vmdemand	VA≠PA remap / demand-paged virtual memory (§6.4)
llm [prompt]	Generate text from the baked-in LLM (§6.3)
preempt	Demo the timer-driven preemptive round-robin scheduler
pingpong [N]	AIPL-style 2-actor cooperative PingPong, N=1..50
procdemo [N]	Real 2-process ctxsw demo, N=1..30
usb	xHCI / DWC2 USB diagnostics (Pi 4 only)
wifi ...	WiFi + mesh: <code>probe/scan/up/adhoc/aadv/...</code> (§7.5–§7.6)
rxstat / tcpstat	RX-ring / TCP-listener counters
pan <dx> <dy> view autopan [on off]	Window-manager viewport controls

6.1 procdemo — real context switch

```
xinu-pi4$ procdemo 3
procdemo: created pid=1 (ping) and pid=2 (pong), iters=3
  [Ping pid=1] tick 1
  [Pong pid=2] tock 1
  [Ping pid=1] tick 2
  [Pong pid=2] tock 2
  [Ping pid=1] tick 3
  [Pong pid=2] tock 3
  [Ping pid=1] exit at iter 3
  [Pong pid=2] exit at iter 3
procdemo: both processes exited; back in shell.
```

Each `pid=N` is read live from global `currpid`, so the alternation is direct evidence the scheduler flipped contexts. Stacks are not reclaimed on `proc_exit` yet — S1 (clock IRQ) will let the dispatcher reap them.

6.2 pingpong vs procdemo

Aspect	pingpong	procdemo
Actors	Two static (Ping, Pong)	Real procs via <code>proc_create</code>
Switch	Single-stack dispatcher loop	Real ctxsw via <code>ctxsw.S</code>
Terminate	Both inboxes empty	Both call <code>proc_exit()</code>

6.3 On-device tooling (filesystem, C JIT, actors, LLM)

Beyond the demos above, the shell carries a self-contained toolchain — you can write, compile, and run code on the board itself.

- **In-RAM filesystem** — `pwd ls cd mkdir rmdir touch cat write edit rm cp mv tree` operate on a small in-memory tree (volatile; cleared on reboot).
- **C JIT (`cc`)** — `cc <file.c>` compiles a C subset to native AArch64 and runs it in place; the same compiler is reachable over HTTP as `POST /compile` (body = C source).
- **AIPL actors** — `aload <file.c>` loads resident actors; `amsg <actor> <method> [arg]` sends a message. `actordemo` runs a 2-actor ping-pong as real Xinu processes; `selectdemo` shows a guarded receive. HTTP `/actor`, `/send`, `/gc` expose the actor inventory, message send, and the actor-pool GC.
- **Embedded LLM (`llm`)** — a tiny transformer is baked into the image; `llm [prompt]` generates text on-device (also `/chat`).

6.4 Virtual memory (`vmtest` / `vmdemand`)

The kernel runs with the MMU on (identity map) plus a **demand-paged window** at VA `0x80000000..0x80400000` (4 MiB, 512-frame pool):

- **`vmtest`** — map a physical page at a different virtual address and prove the translation (`VA ≠ PA`).
- **`vmdemand`** — touch 64 pages in the demand window; the first run takes `0x40` page faults (one per page) and reads back OK, the second run takes `0x0` faults (already mapped). Check the counter with `/fault`.

Both run locally or remotely, e.g. `curl "http://192.168.3.100/shell?cmd=vmdemand"`.

7 Networking (Pi 4)

The network stack runs on the Pi 4 GENET (BCM2711).

What works: ARP request/reply; ICMP echo (ping); static IP/MAC `192.168.3.100 / d8:3a:dd:a7:fd:bf` (set in `loader/main.c:707` area); raw broadcast/unicast TX; 16-slot RX ring.

7.1 Ping from a Mac

```
sudo arp -s 192.168.3.100 d8:3a:dd:a7:fd:bf
ping 192.168.3.100
```

RTT is 900 ms (rate-limited by the window-manager tick). Quick shell checks: `rxstat` and `ticks`.

7.2 DHCP / TCP status

`system/dhcp_client.c` and `system/tcp_server.c` ship in the tree but their `rx_tick` dispatch is **OFF**: repeated DHCP broadcast TX degrades the GENET TX ring and stalls ICMP, and routing `tcp_handle_packet()` from `rx_tick` stops subsequent ICMP echo replies. Re-enabling is planned in a future sprint (NET-G bisect).

7.3 WiFi (BCM43455)

The Pi 4 has an on-board BCM43455 WiFi chip (separate from the wired GENET above). It is driven entirely from the serial shell (`xinu-pi4$`): bring up the firmware, scan, then connect. It does **not** auto-connect at boot.

Command	What it does
<code>wifi probe</code>	M0/M1 bring-up: download the firmware to the chip (once per boot)
<code>wifi scan</code>	escan for nearby APs
<code>wifi up <ssid> <pass></code>	join (WPA2-PSK) + DHCP; starts the persistent ARP/ICMP responder
<code>wifi join <ssid> <pass></code>	join only (no DHCP)
<code>wifi dhcp</code>	request a DHCP lease
<code>wifi off</code>	radio down / disconnect
<code>wifi ping <a.b.c.d> [count]</code>	ICMP echo client
<code>wifi serve [secs]</code>	run the ARP/ICMP responder for N seconds (default 30)
<code>wifi resolve <host> / wifi web <host></code>	DNS / DNS + HTTP GET
<code>wifi ntp [a.b.c.d]</code>	NTP time client
<code>wifi tftp <ip> <file> / wifi netboot <ip> <file></code>	TFTP fetch / fetch + chainload

```
xinu-pi4$ wifi probe          # download firmware (once per boot)
xinu-pi4$ wifi scan           # see what's around
xinu-pi4$ wifi up MyHome-5G mypassword
wifi up: connected; ARP/ICMP responder is now persistent.
xinu-pi4$ wifi ping 8.8.8.8
```

WiFi does not auto-reconnect after a reboot — re-run `wifi probe` + `wifi up` each boot. `wifi netboot <ip> <file>` fetches a kernel over WiFi and chainloads it (a no-SD-swap update path, like the wired chainload).

7.4 Mesh networking: multiple Xinu nodes (MANET ad-hoc)

Several Pi 4 (and Pi 3) Xinu boards can form a peer-to-peer mesh with **no access point**, using 802.11 IBSS (ad-hoc) mode plus on-demand AODV routing — the same MANET stack used in the drone-HIL demo (Pi 4 + Pi 3 nodes). Each node joins the same ad-hoc cell with a distinct node number:

```
xinu-pi4$ wifi probe # once per boot
xinu-pi4$ wifi adhoc <cell-ssid> [ch] [node]
```

- **<cell-ssid>** — the cell name; **all nodes must use the same name and channel.**
- **[ch]** — 802.11 channel (default 6).
- **[node]** — this node's number (default 1); it gets the static IP **10.0.0.<node>/24**. **Give every node a distinct number.**

Example — a 3-node cell on channel 6:

```
xinu-pi4$ wifi adhoc mesh1 6 1 # -> 10.0.0.1
xinu-pi4$ wifi adhoc mesh1 6 2 # -> 10.0.0.2
xinu-pi4$ wifi adhoc mesh1 6 3 # -> 10.0.0.3
```

Nodes within radio range reach each other directly at **10.0.0.x** (**wifi ping 10.0.0.2**).

Multi-hop (AODV). When a destination is not in direct range, discover a route on demand with **wifi aodv <ip>**:

```
xinu-pi4$ wifi aodv 10.0.0.3
[wifi] === AODV discover 10.0.0.3 ===
[wifi] aodv: RREQ id=1 for 10.0.0.3
[wifi] *** AODV route: 10.0.0.3 via 10.0.0.2, 2 hop ***
```

The minimal AODV module (M13, RFC 3561 core) broadcasts an RREQ (UDP port 654); each in-range node relays it and records a reverse route; the destination answers with an RREP that installs the forward route. Every node relays for its peers automatically (route table: up to 16 entries). IBSS / ad-hoc is independent of the infrastructure **wifi up** mode (run **wifi off** to leave an AP first) and is not restored after reboot — re-run **wifi probe + wifi adhoc** on each node.

7.5 HTTP control plane & remote shell

system/tcp_server.c runs an HTTP server on the wired interface (default port 80, **192.168.3.100**), so the board can be driven entirely over the network:

Route	What it does
GET /shell?cmd=<cmd>	run any shell command, return its output (no stdin)
POST /compile	JIT-compile and run a C program (body = source)
GET /chat	on-device LLM chat
GET /actor /send /gc /jitstats	actor inventory / send / GC / JIT counters
GET /usbdia /pcie-init /pcie-enum /xhci-reset	USB / PCIe bring-up diagnostics
GET /fault /mmio-read /mmio-write /mmio-sweep	fault counters + raw MMIO peek/poke
POST /reboot	BCM2711 watchdog reset
POST /chainload	upload + jump to a new kernel (no SD swap)

Runtime diagnostics use these HTTP counters because **uart_puts** deadlocks in the HTTP-worker context — all PCIe/xHCI bring-up logging is done at boot over serial.

Network kernel update (chainload — no SD swap). **POST /chainload** uploads a new kernel and jumps to it in RAM. A helper script wraps the upload:

```
python3 tools/remote_chainload.py 192.168.3.100 compile/kernel8.img
```

A bad image just needs a power cycle (the SD card is untouched), which keeps the dev loop fast. It needs the HTTP server to be responsive to accept the upload.

8 Running under QEMU

```
cd compile
make qemu          # interactive -- Ctrl-A X to quit
make qemu-smoke    # canned input, writes qemu-smoke.log
```

Under QEMU: `MIDR_EL1 = 0x414fd0b1` (QEMU's published Cortex-A76 part number; real Pi 4 hardware runs Cortex-A72); `halt` cleanly exits because `virt` handles PSCI `SYSTEM_OFF`; no networking, USB, SD, or HDMI.

9 Important config.txt Lines

`sdcard/config_pi4.txt` (copied to the card's `config.txt` by `make install_pi4`):

Line	Meaning
<code>arm_64bit=1</code>	Boot in AArch64
<code>kernel=kernel8.img</code>	Pi 4 kernel name
<code>kernel_address=0x80000</code>	Load address
<code>enable_uart=1</code>	Enable UART
<code>uart_2ndstage=1</code>	Print second-stage bootloader log on UART
<code>dtparam=uart0=on</code>	Route UART0 to GPIO14/15
<code>init_uart_clock=48000000</code>	Pin the PL011 reference clock to 48 MHz
<code>hdmi_force_hotplug=1</code>	Force HDMI hotplug
<code>framebuffer_width=640</code>	Width of firmware-allocated framebuffer
<code>framebuffer_height=480</code>	Height
<code>framebuffer_depth=32</code>	bpp

The KMS overlay (`dtoverlay=vc4-kms-v3d`) is left in — Xinu still consumes the firmware's simple framebuffer via the VC mailbox.

10 USB / Input Device Support

The Pi 4's USB-A ports go through **VL805 (PCIe xHCI)**; a hand-rolled xHCI driver brings up USB keyboards and mice.

- **Use the USB-2.0 (black) ports.** They route through the USB-2.0 hub (Genesys, working TT), so low-speed interrupt-IN split transactions are delivered. The USB-3.0 (blue) ports sit behind the VIA USB3 hub whose path does not deliver the periodic transfer (the cursor won't move).
- The shell's `usb` command is a diagnostic dump for the DWC2 (USB-C OTG) HCD, not a general device-control interface.
- VL805 firmware leaves the PCIe RC in reset on an SD boot, so boot issues a `NOTIFY_XHCI_RESET` mailbox tag (`0x00030058`, `devid 0x100000`) to bring xHCI up (details in `NEXT_SESSION_PI4.md`).

Board	Input	Output
Pi 4	USB-A keyboard / UART0 serial	UART0 serial + HDMI framebuffer
QEMU	qemu stdio (serial)	qemu stdio (serial)

11 Troubleshooting

Symptom	What to check
No output on serial	device name + speed in <code>screen</code> ; TX/RX swapped; GND
Banner appears, then stops	Old <code>kernel8.img</code> on SD — rebuild and recopy
No echo at the prompt	Local echo on the host terminal — keep it OFF
<code>peek</code> returns <code>0xFFFFFFFF</code>	Address not mapped; MMU is flat ID
USB keyboard/mouse dead	Make sure it's on a USB-2.0 (black) port (§11)
Ping fails on Pi 4	<code>sudo arp -s ... done?</code> ; cable; <code>rxstat</code> increments?
<code>make qemu</code> fails	Check <code>qemu-system-aarch64</code> version (≥ 11)
<code>make install</code> permission	It uses <code>sudo cp</code> — provide the sudo password
Reboot loop on power-on	Does <code>kernel=</code> match the file on the card?
Rainbow pattern stays on HDMI	Likely missing ARM64 Image header — rebuild

Generic recovery flow:

```
cd compile && make clean && make pi4
make install_pi4 SDCARD=/Volumes
diskutil eject /Volumes/bootfs
# Reseat the SD card and power-cycle the board.
```

12 Directory Layout (operator view)

```
xinu-rpi4/
├── compile/                # run `make` here
│   ├── Makefile
│   ├── kernel8.img        # Pi 4 (built by `make pi4`)
│   └── kernel_virt.img    # QEMU (built by `make qemu`)
├── sdcard/
│   └── config_pi4.txt      # Pi 4 (copied by install_pi4)
├── README.md              # Developer-facing roadmap + internals
├── USERS_MANUAL_EN.md     # Markdown source of this manual
├── USERS_MANUAL_JA.md     # Japanese version
└── NEXT_SESSION_PI4.md    # Handoff notes for ongoing work
```

13 Recipe Book

From a fresh checkout to a running Pi 4:

```
cd /Users/kodamay/projects/xinu-rpi4/compile
make clean && make pi4
make install_pi4 SDCARD=/Volumes
diskutil eject /Volumes/bootfs
screen /dev/tty.usbserial-XXXX 115200
```

Quick command sanity check (QEMU):

```
cd compile && make qemu
xinu-pi4$ procdemo 5
xinu-pi4$ pingpong 3
xinu-pi4$ halt           # Ctrl-A X to exit QEMU
```

14 References

- Upstream Xinu (32-bit): <https://github.com/yaskodama/xinu-rpi>
- AArch64 boot stub origin: <https://github.com/radlyeel/leex>
- Shell centry pattern origin: <https://github.com/davidxyz/xinuPi>
- Development roadmap (Round 1 plan): under `aice-pi-evolution/experiments/` in the `abclcp-project` repo

15 License

Inherits the BSD-style licenses of upstream Xinu and leex. See `LICENSE` once the source-of-truth file is added.